

IINTS-AF Technical Brief

Competition evidence bundle for EUCYS review

Project: IINTS-AF SDK

Purpose: Explain the SDK architecture, reproducibility model, safety layer, and maintenance evidence.

Audience: Technical EUCYS judges, software reviewers, research mentors.

This PDF was rendered from the markdown report and packaged benchmark artifacts inside the IINTS-AF SDK workspace.

Executive Summary

IINTS-AF is a Python SDK for pre-clinical insulin algorithm research. It turns insulin-control experiments into reproducible study bundles: fixed virtual-patient profiles, fixed scenarios, fixed seeds, explicit baselines, data-quality checks, safety-supervisor traces, and generated reports.

The technical contribution is not only an algorithm. It is the research infrastructure around the algorithm.

Design Goals

Goal	Implementation
Reproducible experiments	CLI workflows, fixed seeds, study protocols, run manifests
Baseline comparison	Candidate controller compared against clinical and engineering baselines
Safety visibility	Supervisor decisions logged with intervention reasons
Data trust	MDMP data contracts, certification reports, realism dashboards
Reviewable outputs	Markdown, JSON, CSV, PNG, PDF artifacts
Edge demonstration	Raspberry Pi, UNO Q, and Jetson workflows for offline demos
Maintainability	Tests, flake8, mypy, docs build, GitHub Actions

High-Level Architecture

The SDK is organized around six cooperating layers.

Layer	Role
core	Patient models, simulator loop, safety checks, algorithms
data	Dataset ingestion, MDMP validation, corruption testing, realism checks
research	Predictor training, dataset preparation, model evaluation
analysis	Study summaries, metrics, plots, EUCYS result packaging
live_patient and jetson	Edge and endurance-test workflows
cli	One interface for reproducible project, data, benchmark, and report commands

The CLI is important because it prevents the experiment from becoming a hidden notebook state. A judge can see the command, rerun it, and inspect the output folder.

The Study Bundle Model

A study bundle is a folder that contains both results and proof of how those results were produced.

Typical contents include:

Artifact	Why it matters
protocol/STUDY_PROTOCOL.md	Human-readable experiment design
protocol/study_design.json	Machine-readable locked design
protocol/study_matrix.csv	Exact run matrix
study_summary.json	Aggregated benchmark results
EUCYS_RESULTS_TABLE.csv	Compact result table
EUCYS_MAIN_FIGURE.png	Main visual evidence
run_manifest.json where available	Hashes and provenance

This matters because insulin-algorithm results are easy to overstate if the protocol is not visible. IINTS-AF treats protocol, data quality, and results as one package.

Safety Supervisor

The safety supervisor is a deterministic layer around the algorithm. It does not try to be a black-box AI model. It enforces hard safety rules and records interventions.

Examples of checks include:

- limiting unsafe insulin output
- preventing insulin-on-board overflow
- responding to sensor dropouts or invalid glucose values
- flagging abnormal inputs before they silently enter the algorithm
- logging when and why a decision was changed

This separation is important. The candidate algorithm can be improved independently, while the supervisor remains a transparent safety gate.

MDMP Data Quality Layer

The MDMP layer asks a basic but often ignored question:

Can this dataset be trusted enough to train or evaluate an insulin algorithm?

MDMP checks data contracts, required columns, value ranges, time continuity, missingness, corruption risk, and realism signals. It also creates certification outputs so a reviewer can see whether a dataset passed, warned, failed, or was skipped.

This is especially important for CGM data because a glucose trace can look numeric while still being physiologically implausible: impossible jumps, missing meals, sensor artifacts, duplicated timestamps, or values outside safe interpretation ranges.

Reproducibility Path

The final EUCYS benchmark is intended to be reproducible from one public workflow:

```
tools/research/run_eucys_final.sh \
  --algo algorithms/example_algorithm.py \
  --output-dir results/eucys_2026 \
  --seeds 1,2,3,4,5,6,7,8,9,10 \
  --no-prepare-ai
```

The helper script deliberately uses the source tree instead of relying on a stale global install. It sets PYTHONPATH=src and routes the run through the Typer CLI endpoint.

Benchmark Matrix

The final benchmark design uses:

Dimension	Value
Patient profiles	6
Scenario families	4
Study arms	3
Algorithms	5
Seeds	10
Total runs	3600

The matrix includes clean certified data, corrupted uncertified data, and a supervisor-off ablation. That structure is stronger than a single attractive demo because it tests both normal and failure-prone conditions.

Edge And Jetson Workflows

The SDK includes edge-oriented workflows because EUCYS demonstrations benefit from visible, physical proof that the system can run outside a developer laptop.

Device path	Purpose
Raspberry Pi	Long-running local study or booth demo
Arduino UNO Q bridge	Hardware-facing demonstration path
Jetson endurance mode	Dedicated stress testing with hardware monitoring

These edge paths do not turn the SDK into a medical device. They show that the research workflow can run locally, offline, and reproducibly.

Why This Is Maintainable

Maintenance is part of the project evidence. The latest local and remote checks after the final evidence refresh included:

Check	Result
python3 -m pytest tests/ -q	369 passed, 4 skipped
flake8 .	passed
mypy src/iints/	passed
mkdocs build	passed
tools/ci/check_mdmp_sync.py	passed
GitHub Actions	Python Package CI, Health Badges, Notebook Bake succeeded

The goal is not just to write code once. The goal is to keep the SDK in a state where another person can inspect, run, and challenge it.

Technical Limitations

The SDK is a research platform, so the limitations are explicit:

- virtual patients are approximations, not replacements for clinical trials
- safety-supervisor rules reduce risk but cannot prove absolute safety
- synthetic and simulated data must be checked against real CGM behavior
- benchmark outcomes depend on scenario design and metric definitions
- edge demos are engineering demonstrations, not patient-use systems

Technical Claim

The strongest technical claim is:

IINTS-AF turns insulin algorithm evaluation into a reproducible software engineering and data-quality workflow, with enough auditability for scientific review.